Case Study 1

AKSTA Statistical Computing

The .Rmd and .html (alternatively the .pdf) should be uploaded in TUWEL by the deadline. Refrain from using explanatory comments in the R code chunks but write them as text instead. Please consider the AI Guidelines as specified in the organizational slides (you can use the table). Points will be deducted if the submitted file is not in a decent form; answers should be answering the concrete questions and not discuss topics in general - please be mindful of our tutors' time and the size of the course.

Table 1: You can use and extend this table to specify your AI usage (if used at all).

AI usage	**Task 1**	**Task 2**	...
Correcting faulty code	no		
Improving existing code	little		
Generating new code	medium		
Layout and formatting	heavy		
Text correction			
...			

1. Ratio of Fibonacci numbers (1 point)

a.

Write two different R functions which return the sequence $r_i = F_{i+1}/F_i$ for $i = 1, \ldots, n$ where F_i is the ith Fibonacci number, once using `for` and once using `while`.

b.

Benchmark the two functions for $n = 200$ and $n = 2000$ (you can use package **microbenchmark** or package **bench** for this purpose). Which function is faster?

c.

Plot the sequence for $n = 100$. For which n does it start to stabilize to a value? What number does the sequence converge to?

2. Gamma function (1 point)

a.

Write a function to compute the following for n a positive integer using the `gamma` function in base R.

$$\rho_n = \frac{\Gamma((n-1)/2)}{\Gamma(1/2)\Gamma((n-2)/2)}$$

b.

Try $n = 2000$. What do you observe? Why do you think the observed behavior happens?

c.

Write an implementation which can also deal with large values of $n > 1000$.

d.

Plot $\rho_n/\sqrt{n}$ for different values of n. Can you guess the limit of $\rho_n/\sqrt{n}$ for $n \to \infty$?

3. Weak law of large numbers (1.5 points)

The law of large numbers states that if you repeat an experiment independently a large number of times and average the result, what you obtain should be close to the expected value. Formally, the weak law of large numbers states that if $X_1, X_2, \ldots X_n$ are independently and identically distributed (iid) random variables with a finite expected value $E(X_i) = \mu < \infty$, then, for any $\epsilon > 0$ $\lim_{n\to\infty} P(|\bar{X}_n - \mu| > \epsilon) = 0$ where $\bar{X}_n = \frac{X_1+X_2+\ldots+X_n}{n}$ is the sample mean (the sample mean converges in probability to the expected value).

a.

- In R, simulate rolling a fair 6-sided die $n = 10\,000$ times using vectorized functions.
- After each roll, compute the cumulative mean of all previous rolls.
- Plot the running average against the number of rolls and comment on the result.

b.

- Replicate the die-rolling experiment $M = 50$ times using a for loop. This will result in 50 paths which you can plot against the number of rolls.
- Plot all 50 paths on the same graph to observe the variation in the running averages.

c.

What is the theoretical expected value of a fair 6-sided die?

d.

Comment on how the running mean of the paths behaves relative to the expected value as n increases.

e.

- Modify the code in b. to use a while loop.
- Stop the loop when the proportion of replications where the difference between the running average and the expected value lies outside the interval $[-\epsilon, \epsilon]$ is less than 0.05. Use $\epsilon = 0.01$.
- How high is n at the end of the loop?

f.

- Repeat the analysis in b, but instead of simulating from a discrete uniform distribution, simulate from a continuous standard cauchy distribution using `rcauchy()`. The (standard) cauchy distribution arises from the ratio of two independent mean zero (standard) normally distributed random variables.
- Plot the running mean against increasing n. Comment on whether the weak Law of Large Numbers seems to hold for this distribution. Explain why or why not.

4. Central limit theorem (1.5 points)

The central limit theorem states that, under certain conditions, the sum of a large number of random variables is approximately normal. Consider the case where $X_1, X_2, \ldots, X_n$ are i.i.d. random variables with finite expected values ($E(X_i) = \mu < \infty$) and variances ($Var(X_i) = \sigma^2 < \infty$). The central limit theorem states that the cumulative distribution function (CDF) of the normalized random variable

$$Z_n = \frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}}$$

converges to the standard normal CDF as $n \to \infty$.

a.

Derive the expected value of the sample mean $\bar{X}_n = \frac{X_1+X_2+\ldots+X_n}{n}$.

b.

Derive the variance of the sample mean $\bar{X}_n = \frac{X_1+X_2+\ldots+X_n}{n}$.

c.

The central limit theorem makes few assumption on the distribution of the random variables X_i. Consider again the die-rolling experiment where X_i is the random variable giving the number in roll i.

- Simulate Die Rolls: For each value of n (where $n = \{100, 1000, 10000\}$), simulate rolling a die n times. Record the mean of the n rolls.

- Repeat the above process $M = 100$ times for each value of n. This will give you 100 sample means for each n.
- Plot a histogram of the 100 sample means for each value of n. Observe the shape of the histograms.
- Discuss how the distribution of the sample means changes as n increases. Comment on whether the histograms appear to approach a normal distribution as n, illustrating the Central Limit Theorem.

d.

Compute expected value μ and the variance σ^2 for the random variable X_i (giving the number in die-roll i), which follows a discrete uniform distribution.

e.

Now that you computed μ and σ^2, consider Z_n.

- For each value of n (where $n = \{100, 1000, 10000\}$) and for $M = \{100, 1000, 10000\}$ replications, simulate the rolling of n dice and calculate the Z_n.
- Plot histograms of Z_n for each combination of n and M.
- Discuss how the distribution of Z_n changes as n and M increase. Comment on whether the distributions of Z_n appear to approach a normal distribution, illustrating the Central Limit Theorem.

5. Readable and efficient code (1.5 points)

Read over the code below.

a.

Explain (in text) what the code does.

b.

Explain (in text) what you would change to make the code more readable.

c.

Change the code according to b. and wrap it in a function. This function should have at most 10 lines (without adding commands to more lines such as `x <- 1; y <- 2`. Such commands will count as 2 lines!). Check that the function called on the same input outputs the same as the provided code.

```
set.seed(1)
x <- rnorm(100)
z <- rnorm(100)
if (sum(x >= .001) < 1) {
  stop("step 1 requires 1 observation(s) with value >= .001")
}
```

```r
fit <- lm(x ~ z)
r <- fit$residuals
x <- sin(r) + .01
if (sum(x >= .002) < 2) {
  stop("step 2 requires 2 observation(s) with value >= .002")
}
fit <- lm(x ~ z)
r <- fit$residuals
x <- 2 * sin(r) + .02
if (sum(x >= .003) < 3) {
  stop("step 3 requires 3 observation(s) with value >= .003")
}
fit <- lm(x ~ z)
r <- fit$residuals
x <- 3 * sin(r) + .03
if (sum(x >= .004) < 4) {
  stop("step 4 requires 4 observation(s) with value >= .004")
}
fit <- lm(x ~ z)
r <- fit$residuals
x <- 4 * sin(r) + .04
x
```

```r
set.seed(1)
x <- rnorm(1000)
y <- 2 + x + rnorm(1000)
df <- data.frame(x, y)

cat("Step", 1, "\n")
fit1 <- lm(y ~ x, data = df[-(1:250),])
p1 <- predict(fit1, newdata = df[(1:250),])
r <- sqrt(mean((p1 - df[(1:250),"y"])^2))

cat("Step", 2, "\n")
fit2 <- lm(y ~ x, data = df[-(251:500),])
p2 <- predict(fit2, newdata = df[(251:500),])
r <- c(r, sqrt(mean((p2 - df[(251:500),"y"])^2)))

cat("Step", 3, "\n")
fit3 <- lm(y ~ x, data = df[-(501:750),])
p3 <- predict(fit3, newdata = df[(501:750),])
r <- c(r, sqrt(mean((p3 - df[(501:750),"y"])^2)))

cat("Step", 4, "\n")
fit4 <- lm(y ~ x, data = df[-(751:1000),])
p4 <- predict(fit4, newdata = df[(751:1000),])
r <- c(r, sqrt(mean((p4 - df[(751:1000),"y"])^2)))
r
```

6. Measuring and improving performance (1.5 points)

Have a look at the code of the function below. It is a function for performing a Kruskal-Wallis test, a robust non-parametric method for testing whether samples come from the same distribution. (Note: we assume no missing values are present in `x`).

```
kwtest <- function (x, g, ...)
{
    if (is.list(x)) {
        if (length(x) < 2L)
            stop("'x' must be a list with at least 2 elements")
        if (!missing(g))
            warning("'x' is a list, so ignoring argument 'g'")
        if (!all(sapply(x, is.numeric)))
            warning("some elements of 'x' are not numeric and will be coerced to numeric")
        k <- length(x)
        l <- lengths(x)
        if (any(l == 0L))
            stop("all groups must contain data")
        g <- factor(rep.int(seq_len(k), l))
        x <- unlist(x)
    }
    else {
        if (length(x) != length(g))
            stop("'x' and 'g' must have the same length")
        g <- factor(g)
        k <- nlevels(g)
        if (k < 2L)
            stop("all observations are in the same group")
    }
    n <- length(x)
    if (n < 2L)
        stop("not enough observations")
    r <- rank(x)
    TIES <- table(x)
    STATISTIC <- sum(tapply(r, g, sum)^2/tapply(r, g, length))
    STATISTIC <- ((12 * STATISTIC/(n * (n + 1)) - 3 * (n + 1))/(1 -
        sum(TIES^3 - TIES)/(n^3 - n)))
    PARAMETER <- k - 1L
    PVAL <- pchisq(STATISTIC, PARAMETER, lower.tail = FALSE)
    names(STATISTIC) <- "Kruskal-Wallis chi-squared"
    names(PARAMETER) <- "df"
    RVAL <- list(statistic = STATISTIC, parameter = PARAMETER,
        p.value = PVAL, method = "Kruskal-Wallis rank sum test")
    return(RVAL)
}
```

a.

Write a pseudo code outlining what the function does.

b.

For example data, call the function in two ways: once using `x` as a list and once using `x` as a vector with a corresponding `g` argument. Ensure that the two different function calls return the same thing by aligning the inputs.

c.

Make a faster version of `kwtest()` that only computes the Kruskal-Wallis **test statistic** when the input is a numeric variable `x` and a variable `g` which gives the group membership. You can try simplifying the function above or by coding from the mathematical definition (see https://en.wikipedia.org/wiki/Kruskal%E2%80%93Wallis_one-way_analysis_of_variance). This function should also perform some checks to ensure the correctness of the inputs (use `kwtest()` as inspiration).

d.

Consider the following scenario. You have samples available from multiple experiments $m = 1000$ where you collect the numerical values for the quantity of interest `x` and the group membership for n individuals. The first 20 individuals in each sample belong to group 1, the following 20 individuals in each sample belong to group 2, the last 10 individuals in each sample belong to group 3. Use the following code to simulate such a data structure:

```
set.seed(1234)
m <- 1000 # number of repetitions
n <- 50   # number of individuals
X <- matrix(rt(m * n, df = 10), nrow = m)
grp <- rep(1:3, c(20, 20, 10))
```

Write a function which performs the Kruskal-Wallis test using the function `stats:::kruskal.test.default()` for m repeated experiments and returns a vector of m test statistics. The input of this function are a matrix `X` with m rows and n columns and a vector `g` which gives the grouping.

e.

Write a function which performs the Kruskal-Wallis test using the function in point c. for m repeated experiments and returns a vector of m test statistics. The input of this function are a matrix `X` with m rows and n columns and a vector `g` which gives the grouping.

f.

Compare the performance of the two approaches using a benchmarking package on the data generated above. Comment on the results.

g.

Now consider vectorizing the function in point c. Compare this approach to the other two. Comment on the results.